Gravity-Bot

Rapport 3

Groupe CD3

Abid Mohamed, El Amrany Anas, El Haddadi Bilele, El Hauat Chada, El Qabbab Oussama, Hosayn Mohamed Anis, Kibou Yassine, Masbah Louay, Samari Yassine

Table des matières

Introduction 3
Sujet du projet 3
Fonctionnalités 3
Paquetages 3
Paquetage core 4
Paquetage interface 5
Organisation 6
Problèmes rencontrès 6
Développement 6
Boucle de jeu bloquante 6
Bloquage du projectile 6
Organisation 7
Gestion du git 7
Note sur l'utilisation d'IA générative 7

Introduction

Sujet du projet

Notre projet long porte sur le développement d'un jeu multijoueur d'arcade en Java et Swing. L'objectif est d'affronter son adversaire sur une map 2D. La map contient des obstacles, un satellite et un objectif à atteindre. Grâce à son budget, le joueur peut placer des obstacles pour modifier la trajectoire du satellite. Le gagnant est le dernier qui obtient une trajectoire corrects. Les obstacles de l'adversaires sont conservées lors du tour.

Fonctionnalités

- En tant que joueur, je peux choisir la map sur laquelle je vais affronter mon adversaire.
- En tant que joueur, je peux placer des obstacles sur la Map 2D affiché sur mon écran.
- En tant que joueur, je dois voir et gérer mon budget pour le placement d'obstacles.
- En tant que joueur, je peux créer des maps, les essayer, et les jouer contre un joueur.

Fonctionnalités manquantes :

- En tant que joueur, je peux jouer contre l'IA.
- En tant que joueur, je ne peux pas faire de l'anti-jeu.
- En tant que joueur, je peux customiser mon satellite.

Paquetages

Nous avons décidés de découper le projet en 2 paquetages. Nous suivons une conception MVC : Modèle, Vue, Contrôleur.

- **Core** : contient le modèle. Peut fonctionner sans interface. Le jeu est géré dans `Game` où sont enregistré les joueurs et les tours. La `Map` contient les obstacles. Y sont développé aussi les obstacles.
- **Interface** : contient l'IHM et les différents controleurs (éditeur de niveau, jeu).
- **Utils** : Classes utiles pout tout les paquetages comme la classe `Vecteur`. (implémentation très courte).

Paquetage core

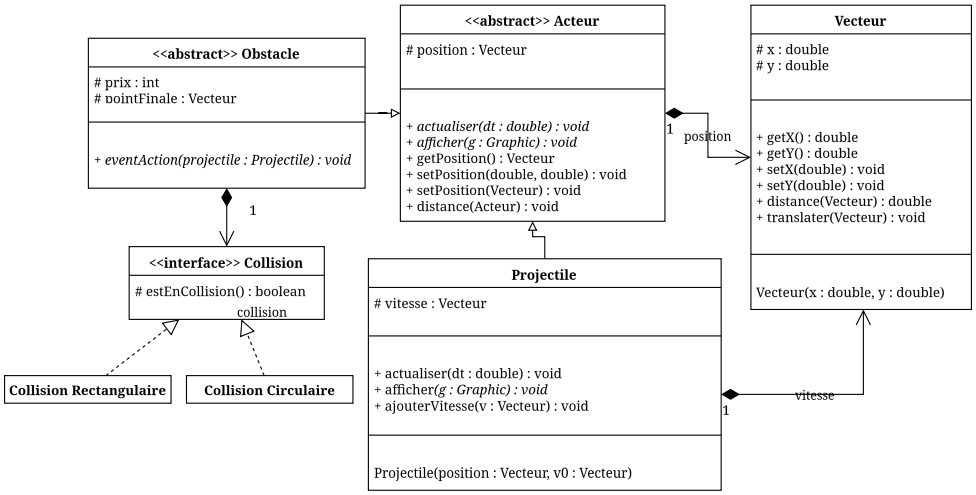


Fig. 1. – Diagramme UML partiel des acteurs.

Nous avons fait le choix de développer une classe abstraite `Acteur`. Tout élément possédant des coordonnées spatiales sur le plan hérite de cette classe. Pour optimiser le cycle de rendu, chaque acteur est responsable de son propre comportement d'affichage en implémentant une méthode dédiée prenant en paramètre le contexte `Graphics` de Java.

Graphics permet de placer des formes géométriques sur la map. Cela donne une direction artistique simpliste et légère (pas besoin d'images) que nous appréciions.

Le `Projectile` est le seul qui peut se déplacer. C'est pourquoi c'est le seul qui possède un attribu vitesse. Ce dernier est actualisé par la classe `Map` toutes les frames. Le `dt` permet de ne pas prendre en compte les varitations de temps entre les frames.

$$P_{t+1} = P_t + (V \times dt)$$

Pour des raisons d'évolutivité, nous avons opéré un découpage strict entre la nature d'un `Obstacle` et sa forme géométrique de collision (`Collision`). Ainsi, un obstacle peut se voir attribuer dynamiquement un `CollisionRectangle` ou un `CollisionCirculaire` sans altérer son comportement intrinsèque, favorisant une réutilisation maximale du code de détection.

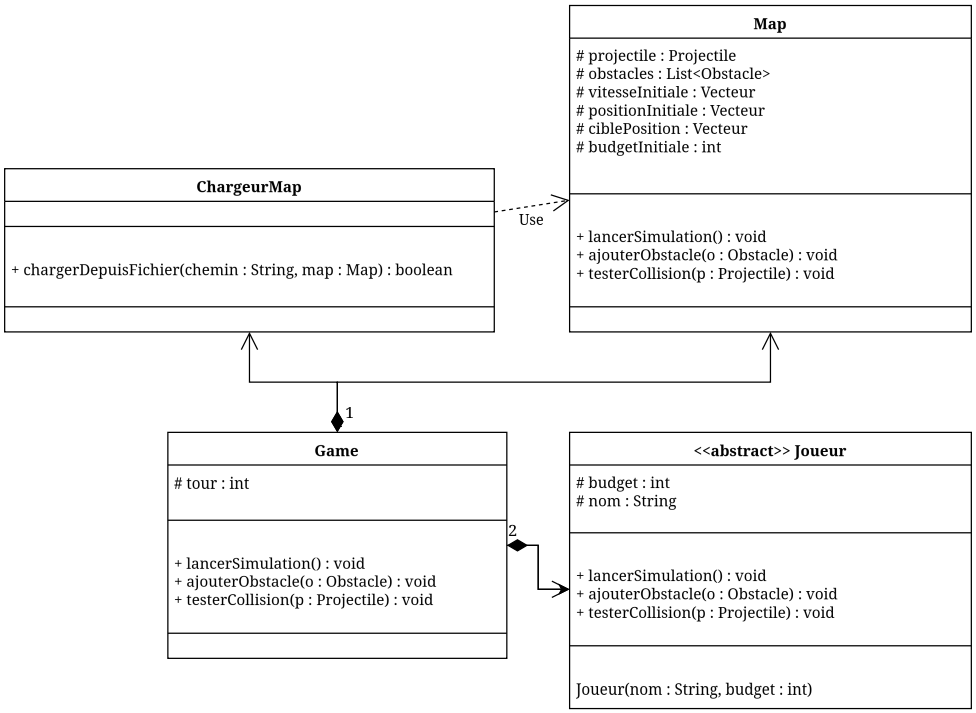


Fig. 2. – Diagramme UML partiel des acteurs présents dans la Map.

La classe `Map` centralise la boucle de simulation. À chaque frame, elle interroge le sous-système de collision pour vérifier si le `Projectile` intersecte la hitbox d'un `Obstacle`. En cas d'événement positif, la méthode `eventAction(Projectile p)` de l'obstacle concerné est déclenchée. L'obstacle reçoit ainsi une référence directe vers le projectile, lui permettant d'altérer instantanément son vecteur de vitesse ou sa position (comme l'inversion vectorielle lors d'un rebond ou l'accélération gravitationnelle).

Pour simplifier la création de maps, nous avons crée une classe `ChargeurMap` qui lit le contenue d'un fichier texte et ajoute les obstacles lues. Ce chargement s'effectue par `Game` lors du lancement de la partie.
Nous avons du faire appel à une classe `ObstaclesRegistre` qui enregistre les obstacles qui peuvent être placés. Une liste associe un nom (présent dans le fichier) à un lambda qui crée l'obstacle à l'aide des paramètres données.

Paquetage interface

L'interface à été implémenté avec `Swing`.

Les contrôleurs enregistrent les event possibles pour l'utilisateur. Il est plus facile ainsi de se retrouver dans le code de l'interface.

- ControleurJeu :
  - Cliquer et restez appuyé permet de placer un obstacle avec une direction.
  - Bouton lancer simulation.

- ControleurEditeur:

  – Placement sur une grille des obstacles.

La classe `MenuPrincipale` est le point d'entrée de l'utilisateur.

La classe `SelectionMap` liste dynamiquement les fichiers .txt du répertoire `src/core/maps/`. Cette interface permet de choisir manuellement une carte ou d'en générer une aléatoirement via un bouton dédié, avant de lancer la partie avec le constructeur adapté

Pour le jeu, nous avons développer l'interface `FenetreJeu` qui affiche :

– Une sidebar avec une liste des obstacles (nom et coût) que nous devons mettre à jour pour ajouter des obstacles.

– Des labels qui affiche le nom du joueur, le tour et le budget.

Organisation

Un groupe à travaillé sur **interface** : El Amrany Anas, El Hauat Chada, Kibou Yassine, Masbah Louay.

Un autre groupe à travaillé sur **core** : Abid Mohamed, El Haddadi Bilele, El Qabbab Oussama, Hosayn Mohamed Anis, Masbah Louay, Samari Yassine.

Nous avons crée l'UML de **core** avant le début du projet. Ainsi, dès que le groupe **core** à terminé son implémentation, il fut facile de le connecter à l'interface.

De plus, nous avons rempli le GitHub project avec les user stories. Les tâches étaient triés dans l'ordre de priorité (Backlog). Durant chaque sprint, nous savons qui implémente quoi en nous assignant sur la task.

Problèmes rencontrès

Développement

Boucle de jeu bloquante

L'implémentation actuelle de notre simulation est bloquante. Il est impossible d'interagir avec l'interface durant la simulation. Une solution aurait été d'utiliser un autre thread. Toutefois, cette méthode apporte beaucoup de difficultés sur la manière dont les données sont utilisées. Nous avons donc fait le choix de garder une boucle simple bloquante.

Bloquage du projectile

Dans certaines situations, il était possible de bloquer le satellite (ou de le rendre très lent). Puisqu'il n'est pas possible de stopper la simulation, cela crée un blocage.

Nous avons donc ajouté une limite minimale de vitesse pour éviter cela. Toutefois, il est toujours possible de bloquer le projectile autrement.

Organisation

Gestion du git

La gestion d'un dépôt Git partagé à 9 collaborateurs représente un défi logistique important. Par simplicité, nous avons préféré ne pas utiliser de branches ni de pull requests. Tout le monde travaillait sur main. Cela s'est avéré être une erreur et nous a fait perdre du temps.

Note sur l'utilisation d'IA générative

Nous avons décidé de recourir à l'intelligence artificielle pour certaines tâches tout en restant conforme aux attentes du projet. Toutes les parties où elle a été utilisée, partiellement ou totalement, ont été marquées dans les rapports. Les parties concernées ont été comprises et vérifiées par les membres du groupe. Voici un résumé de son utilisation :

- Certains affichages d'acteurs avec `Graphics` pour obtenir un beau résultat.
- Le design du Menu principal (déjà présent mais non estétique).